

ADR 0001 — Arquitectura de datos móvil

Estrategia **offline-first híbrida** · Pawcare — App móvil para atención veterinaria a domicilio

Aceptado 2026-05-31 Expo SDK 56 · React Native · TypeScript SQLite (expo-sqlite + Drizzle) redux-persist

API REST Rails

Contexto y decisión

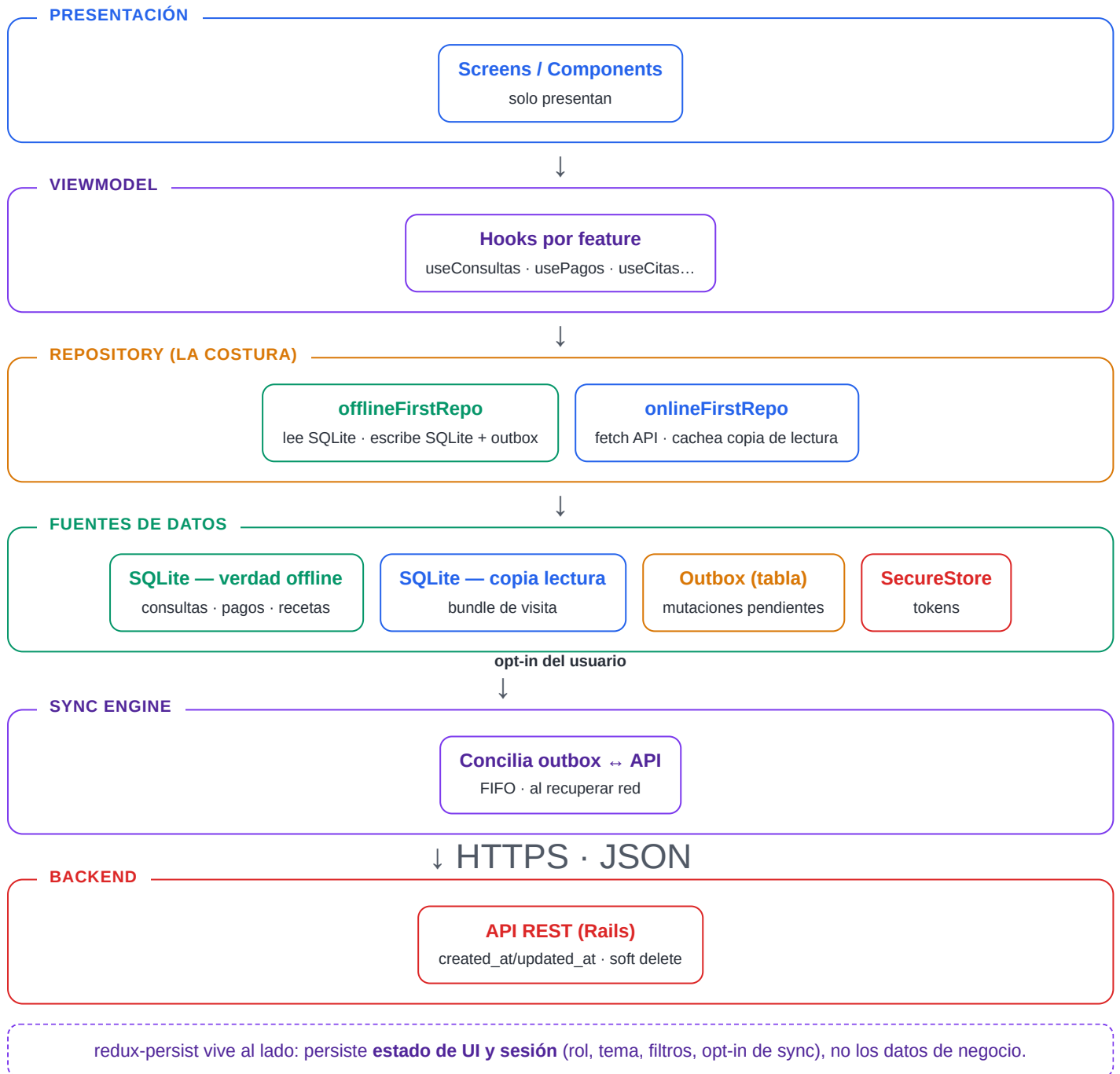
La app habilita **atención veterinaria a domicilio**: la conexión es mala justo donde ocurre el trabajo (la casa del paciente). No todo necesita operar sin red, pero los flujos de la visita —**registrar la consulta** y **registrar el pago recibido**— deben funcionar offline. Se adopta una arquitectura por capas con **Repository pattern** como costura entre la UI y las fuentes de datos, con una clasificación de datos en tres niveles.

Regla de oro: no duplicar el mismo dato. **SQLite** = datos relacionales/consultables/grandes + verdad offline + copias de lectura + *outbox*. **redux-persist** = estado de UI y server-state pequeño (sesión, tema, filtros, opt-in). **SecureStore** = tokens y secretos.

Este documento explica, con diagramas: (2) las capas de datos, (3) la clasificación en 3 niveles, (4) el "bundle de visita" y la escritura offline, (5) el flujo del outbox/sincronización y (6) el reparto de almacenamiento.

1. Capas de datos

La UI nunca llama al API directamente: pide al repositorio, que decide local vs remoto.



2. Clasificación de datos en tres niveles

Qué se puede escribir offline, qué solo se cachea para lectura y qué exige conexión.

Nivel 1 · Offline-first

SQLite (fuente de verdad) + outbox · escritura offline

Consultas

Recetas

Vacunas

Desparasitaciones

Exámenes lab

Adjuntos médicos

Pagos recibidos ★

Estado de cita (en visita)

Borradores

Nivel 2 · Online-first + copia

API; copia de solo-lectura en SQLite; escritura requiere red

Citas (listado)

Pagos (listado)

Productos

Dashboard

Perfil

Pacientes / Owners

Servicios

Bundle de visita

Nivel 3 · Online-only

Sin valor offline; no se cachea

Login / Registro

Recuperar contraseña

Generar reportes

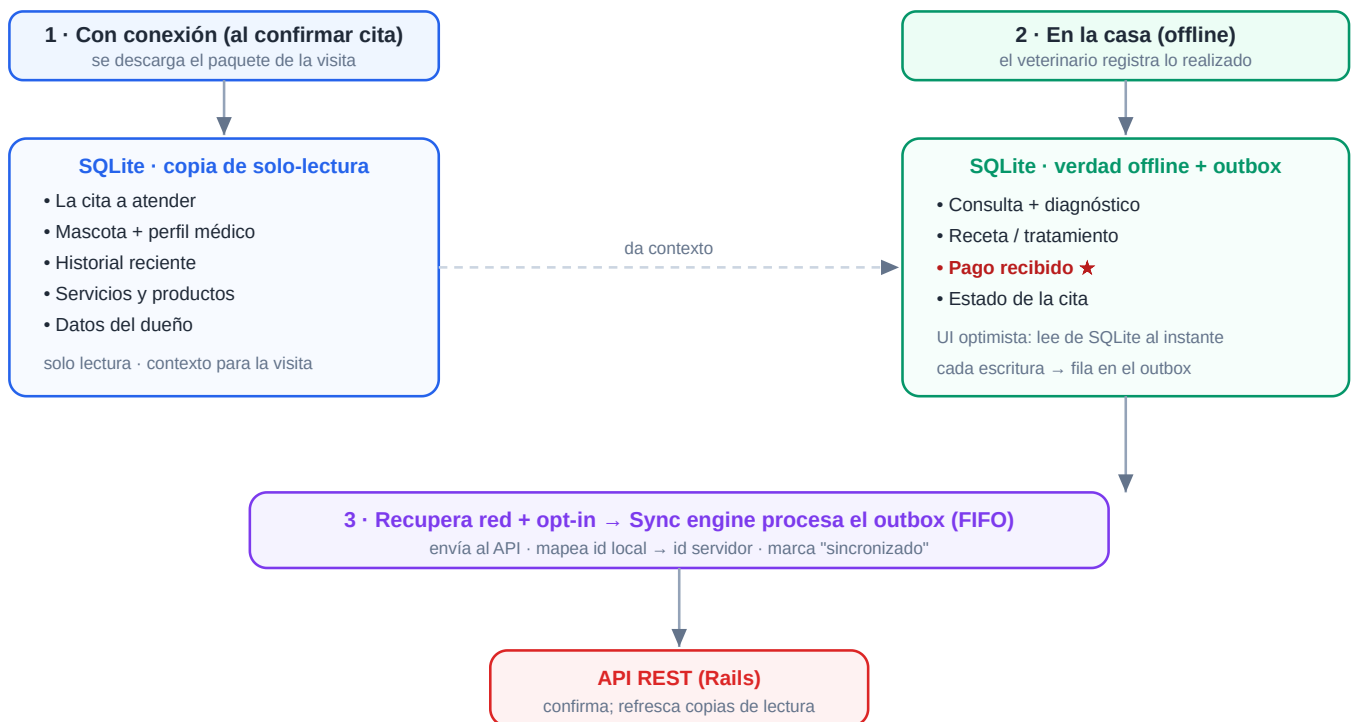
Exportar (CSV/PDF/JSON)

Enviar por email

★ El registro de pago recibido fuera de la clínica es offline-first, igual que la consulta: el cobro ocurre en la casa.

3. "Bundle de visita" y escritura offline

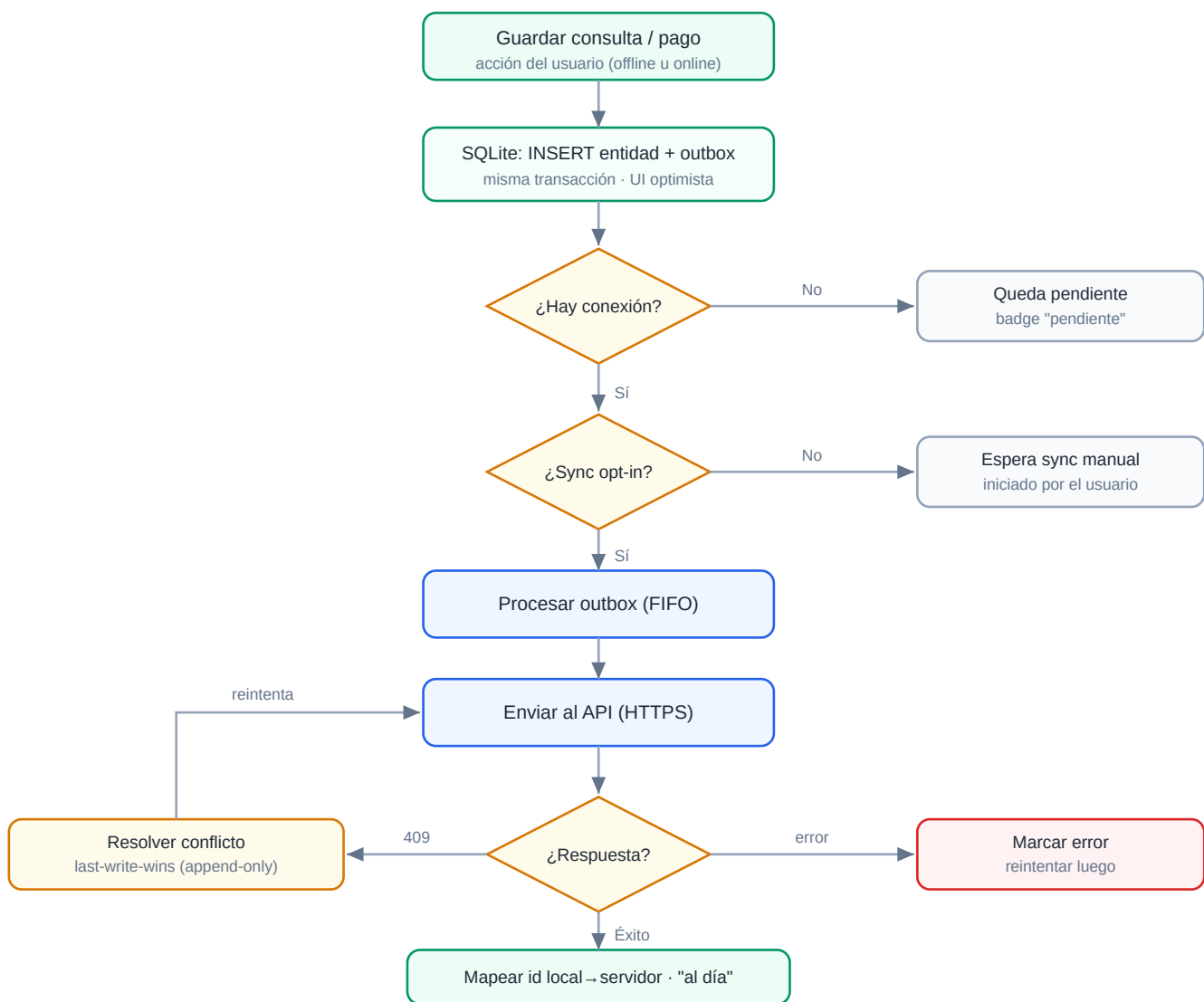
Antes de la visita se replica a SQLite solo lo necesario (lectura). En la casa se crea consulta y pago (verdad local + outbox).



Hasta que el outbox sincroniza, la consulta y el pago se muestran como **"pendientes de sincronización"**; no se simula éxito en el servidor.

4. Flujo del outbox y la sincronización

Mismo camino para una consulta o un pago recibido. Rombos = decisión.



5. Reparto de almacenamiento

Cada almacén tiene un trabajo distinto. La disciplina evita duplicar datos.



Si lo vas a consultar/filtrar o es grande → **SQLite**. Si es estado de pantalla pequeño → redux-persist. Si es un secreto → SecureStore.